638. Shopping Offers

Medium | Topics | Companies

In LeetCode Store, there are `n` items to sell. Each item has a price. However, there are some special offers, and a special offer consists of one or more different kinds of items with a sale price.

You are given an integer array `price` where `price[i]` is the price of the `i^th` item, and an integer array `needs` where `needs[i]` is the number of pieces of the `i^th` item you want to buy.

You are also given an array `special` where `special[i]` is of size `n + 1` where `special[i][j]` is the number of pieces of the `j^th` item in the `i^th` offer and `special[i][n]` (i.e., the last integer in the array) is the price of the `i^th` offer.

Return *the lowest price you have to pay for exactly certain items as given, where you could make optimal use of the special offers*. You are not allowed to buy more items than you want, even if that would lower the overall price. You could use any of the special offers as many times as you want.

Example 1:

```
Input: price = [2,5], special = [[3,0,5],[1,2,10]], needs = [3,2]
Output: 14
Explanation: There are two kinds of items, A and B. Their prices are $2 and $5 respectively.
In special offer 1, you can pay $5 for 3A and 0B
In special offer 2, you can pay $10 for 1A and 2B.
You need to buy 3A and 2B, so you may pay $10 for 1A and 2B (special offer #2), and $4 for 2A.
```

Example 2:

```
Input: price = [2,3,4], special = [[1,1,0,4],[2,2,1,9]], needs = [1,2,1]
Output: 11
Explanation: The price of A is $2, and $3 for B, $4 for C.
You may pay $4 for 1A and 1B, and $9 for 2A ,2B and 1C.
You need to buy 1A ,2B and 1C, so you may pay $4 for 1A and 1B (special offer #1), and $3 for 1B, $4 for
1C.
You cannot add more items, though only $9 for 2A ,2B and 1C.
```

```c
#include <limits.h>

int min(int a, int b) { return a < b ? a : b; }
int dfs(int* price, int** special, int specialSize,
        int* needs, int n) {

    int cost = 0;

    for (int i = 0; i < n; i++)
        cost += price[i] * needs[i];

    for (int i = 0; i < specialSize; i++) {
        int newNeeds[n];
        int valid = 1;

        for (int j = 0; j < n; j++) {
            if (special[i][j] > needs[j]) {
                valid = 0;
                break;
            }
            newNeeds[j] = needs[j] - special[i][j];
        }

        if (valid) {
            int offer = special[i][n];
            cost = min(cost, offer +
                       dfs(price, special, specialSize, newNeeds, n));
        }
    }

    return cost;
}

int shoppingOffers(int* price, int priceSize,
                   int** special, int specialSize,
                   int* specialColSize,
                   int* needs, int needsSize) {

    return dfs(price, special, specialSize, needs, needsSize);
}
```

Accepted Runtime: 0 ms

Case 1 Case 2

Input

price =

[2,5]

special =

[[3,0,5],[1,2,10]]

needs =

[3,2]

Output

14

Expected

14

Accepted Runtime: 0 ms

Case 1 Case 2

Input

price =

[2,3,4]

special =

[[1,1,0,4],[2,2,1,9]]

needs =

[1,2,1]

Output

11

Expected

11